SAKUNA.PH

Centralized Historical Disaster Database for the Philippines

SPIDER-ISKOLARS

Members:

Josh Rylie Patiño

Ace Zeus Rimo

Client Name:

Jake Rom Cadag, Ph.D.

(Department of Geography, UP Diliman)

https://joshdotpatinof.github.io/spider-iskolars-blog/

Software Engineering 1

CS 191 WFR1

Instructor: Ligaya Leah Figueroa, Ph.D.

1. Web App Requirements Model

a. Content Model

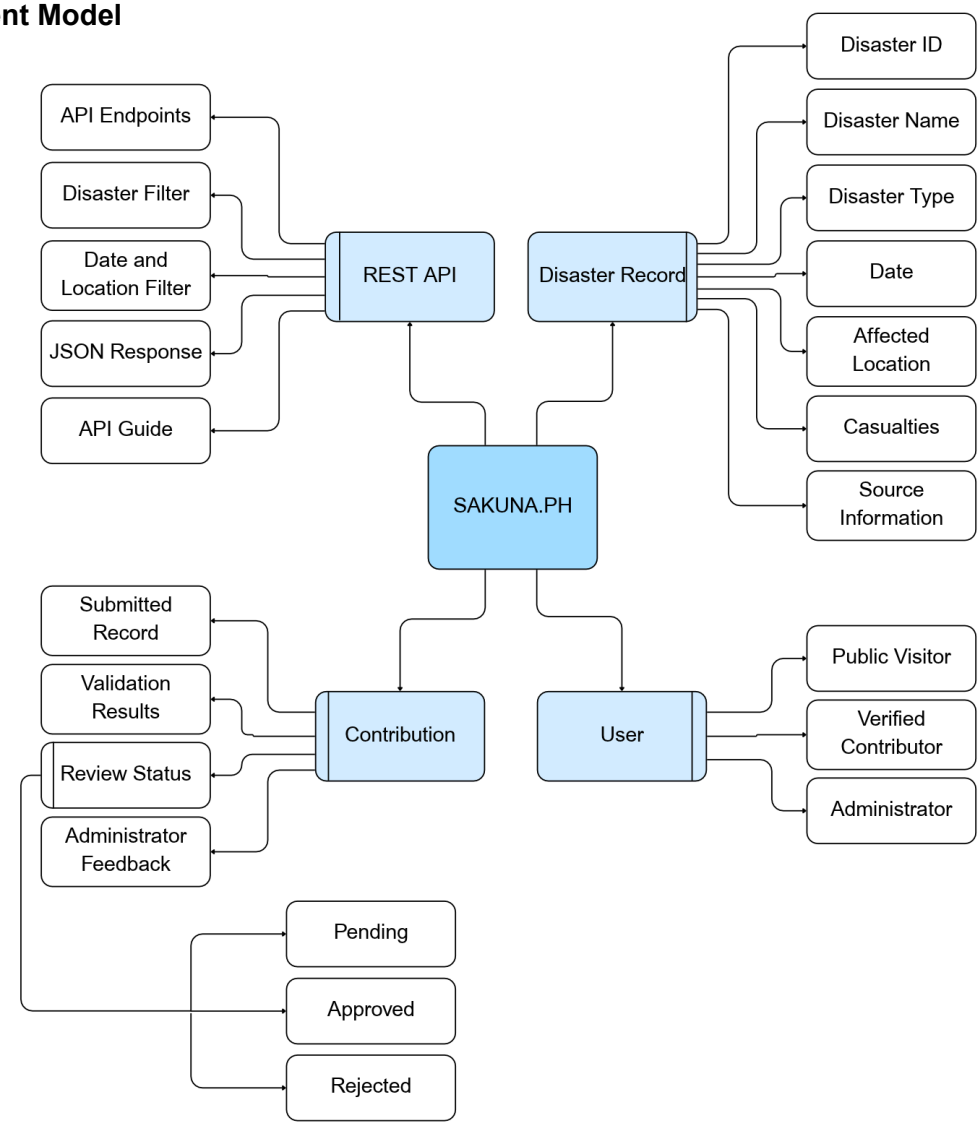


Figure 1: A Content Model of SAKUNA.PH

The content model of SAKUNA.PH illustrates the relationships between its main content objects. The system centers on disaster records, contributions, users, and the REST API. Disaster records contain information such as the disaster ID, name, type, date, affected location, casualties, and source information. Contributions contain submitted records, validation results, review statuses, and administrator feedback, with statuses classified as pending, approved, or rejected. The user object identifies the public visitor, verified contributor, and administrator, while the REST API provides endpoints, filtering options, JSON responses, and an API guide for accessing disaster data. Additionally, the system incorporates four key page

structures: (1) the Contribution Pipeline Page for submitting disaster records for administrative validation, (2) the API Page containing the comprehensive API Guide and endpoint specs for programmatic data retrieval, (3) the Data Science Projects Showcase featuring analytical projects, notebooks, and studies referencing the database, and (4) the Modified Landing Page incorporating summary statistics (total disasters, casualties, and financial damage) alongside trend cards.

b. Interaction Model

i. Actors and Goals

<table>
<tr><th colspan="2">Actors</th></tr>
<tr><td>“Initiating”
(Initiates a use case by initiating an external event, they provide system inputs)</td><td>“Participating”
(Involved in a use case, but does not initiate it, they receive system outputs)</td></tr>
<tr><td>● Public Visitor
● Verified Contributor / Local Official
● Administrator
● External Researcher / API Consumer</td><td>● GraphDB Database Service - Store and retrieve historical disaster records requested by the application.
● External AI Service - Generates chatbot responses using the disaster-related questions and contextual information supplied by the application.</td></tr>
<tr><th colspan="2">Goals</th></tr>
<tr><td colspan="2">● Public Visitor: To browse and search disaster records, view statistics and trends, interact with the AI chatbot, and explore data science showcases.
● Verified Contributor / Local Official: To submit disaster contributions and track the status of submitted records.
● Administrator: To review and verify submitted disaster contributions and maintain the database.
● External Researcher / API Consumer: To retrieve disaster data programmatically through the REST API and consult the API documentation.</td></tr>
</table>

ii. Use Cases

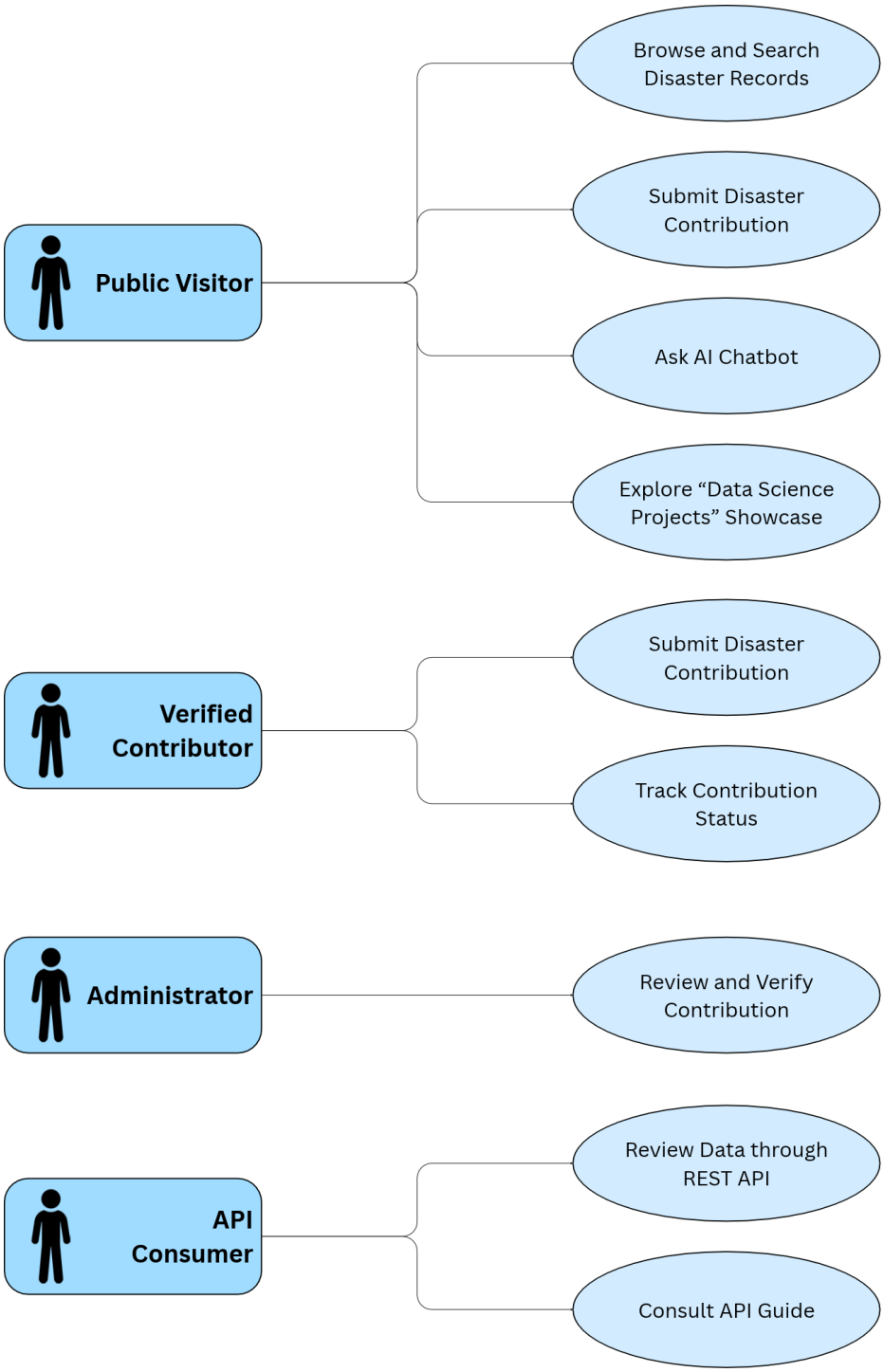


Figure 2: Use Case Diagrams of SAKUNA.PH

Use Case	Description
Browse and Search Disaster Records	Allows users to search, filter, and view historical disaster records stored in the database.
Submit Disaster Contribution	Allows users and verified contributors to submit new disaster records or information for verification.
Ask AI Chatbot	Enables interactive querying about disaster data using an external AI assistant.
Explore "Data Science Projects" Showcase	Provides access to data science analytical projects, visual representations, and insights derived from disaster data.
Track Contribution Status	Allows contributors to view the review and approval status of their submitted disaster records.
Review and Verify Contribution	Enables administrators to inspect, edit, approve, or reject user-submitted disaster reports.
Review Data through REST API	Allows API consumers and researchers to programmatically query and retrieve disaster datasets.
Consult API Guide	Provides documentation, endpoints, and usage instructions for interacting with the REST API.

iii. System Sequence Diagrams

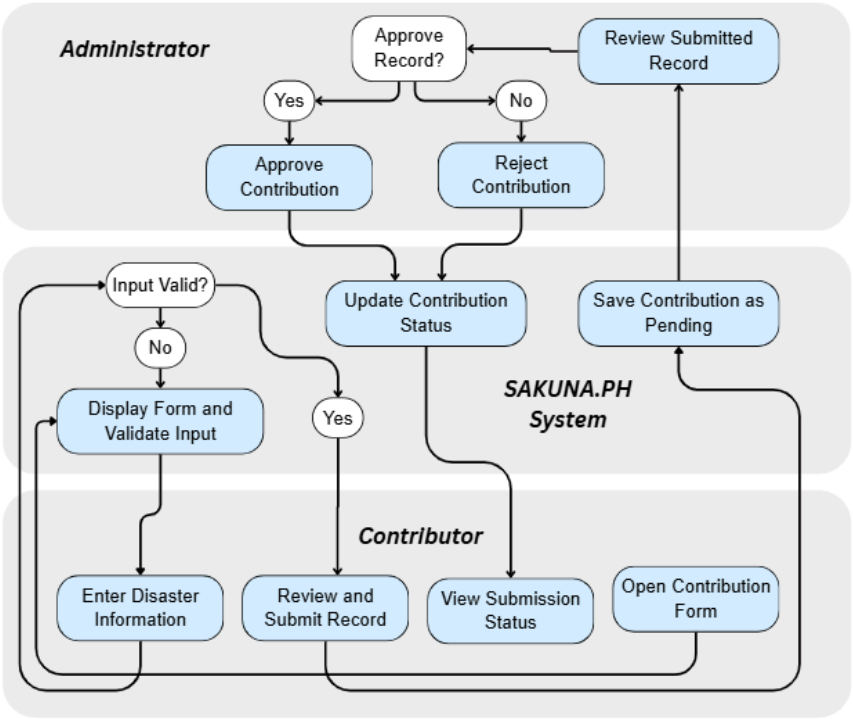


Figure 3: Sequence Diagram of Contribution Pipeline

The sequence diagram illustrates the contribution pipeline of SAKUNA.PH, showing how contributors submit disaster records for administrative review. The system validates the input and saves valid submissions as pending, allowing administrators to review and either approve or reject each record. The system then updates the contribution status, which contributors can view to track the outcome of their submission.

c. Configuration Model

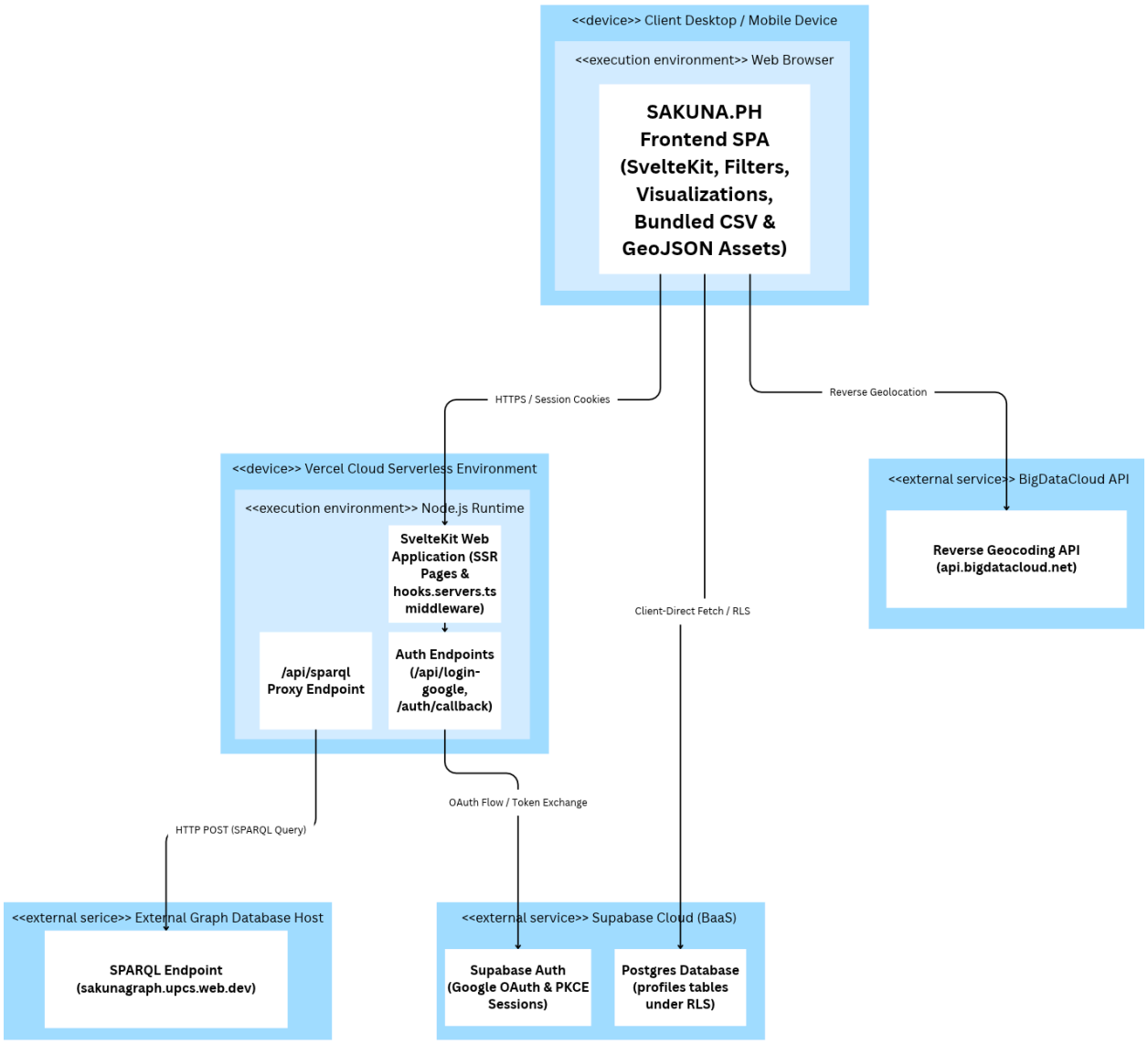


Figure 4: UML Deployment Diagram for SAKUNA.PH WebApp

The SAKUNA.PH web application operates across a serverless and distributed cloud architecture designed to handle web traffic, perform secure authentication, execute client-side visualizations, and securely query external graph database and BaaS services. Users interact with the platform via standard web browsers on desktop and mobile devices, utilizing a client-side single-page application built with SvelteKit that handles user interface rendering, reactive data filtering, map visualizations, form submissions, bundled static datasets such as disaster_data.csv and GeoJSON boundary assets, direct integration with the Supabase database via Row-Level Security, and reverse geolocation through the BigDataCloud API. The backend environment runs on a scalable serverless Node.js execution environment managed by Vercel, featuring a SvelteKit web application for server-side rendering, page routes, and session middleware, along with specific auth endpoints managing OAuth login flows and token exchanges, and an /api/sparql proxy endpoint acting as a secure server-side proxy to forward queries to the external graph database. Finally, the system integrates specialized external cloud providers, including Supabase Cloud for user authentication and hosting the PostgreSQL profiles table, an external graph database host maintaining the historical disaster graph ontology via a public SPARQL endpoint, and the BigDataCloud API for client-called reverse geocoding services.

2. Web App Architecture and Design

The architecture and component-level design of SAKUNA.PH translate the functional and configuration requirements into executable software constructs. The design guarantees scalability across high-volume SPARQL graph queries, efficient AI context management, and intuitive user interaction.

a. Architecture Design

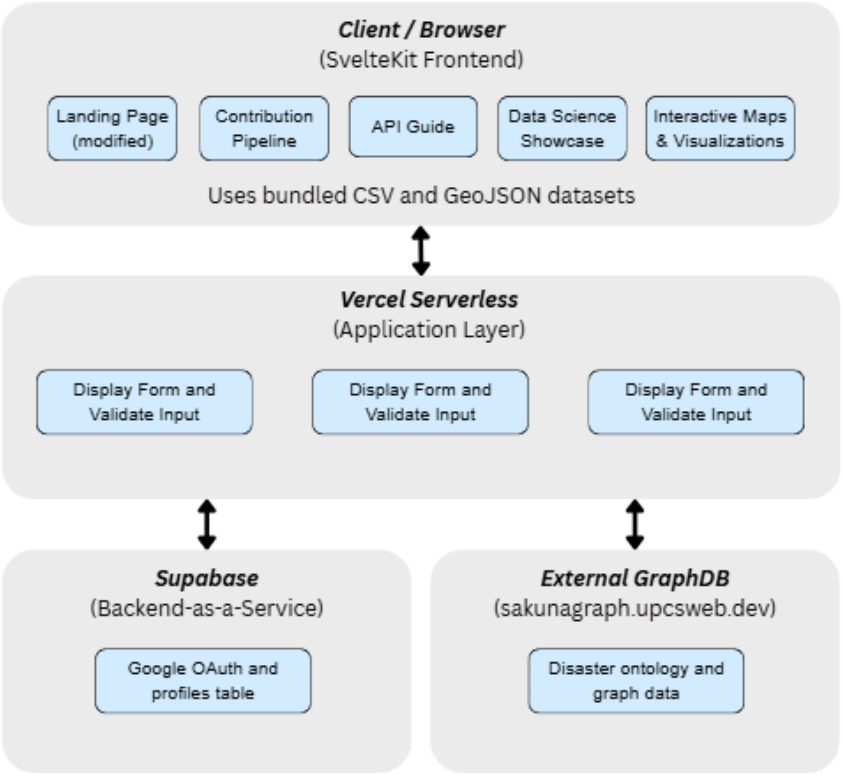


Figure 5: Architecture Design of the WebApp with the Added Features

The revised SAKUNA.PH architecture follows a client-server model with a serverless application layer. The SvelteKit frontend runs in the user's browser and provides the landing page, Contribution Pipeline, API Guide, Data Science Projects Showcase, and interactive maps and visualizations. The application is deployed through Vercel's serverless runtime, which handles server-side rendering, session middleware, and requests to the external graph database through the /api/sparql proxy endpoint. Supabase provides Google OAuth authentication and profile management, while the external GraphDB service stores the disaster ontology and graph data.

The architecture supports the system's quality requirements through a consistent interface for usability, dedicated pages for the added features, and a server-side proxy that

organizes communication with the external graph database. Using bundled CSV and GeoJSON datasets also allows the frontend to display disaster information and visualizations. The separation between the frontend, serverless application layer, and external services helps keep the system organized and makes individual components easier to maintain and extend.

b. Content (Data) Design

i. Class Diagrams

The diagram below presents the proposed class diagram for the planned SAKUNA PH enhancements. The Contribution, VerificationReview, and ContributionPipeline classes support disaster data submission and verification. The Dashboard and DataFilter classes handle disaster statistics and trends, while DataScienceShowcase and DatasetResource support dataset browsing and downloads. Meanwhile, APIEndpoint and APIGuide represent the Open Data Gateway and its documentation.

The diagram illustrates the classes' attributes, operations, and dependencies, providing a conceptual overview of how the planned features will interact within SAKUNA PH.

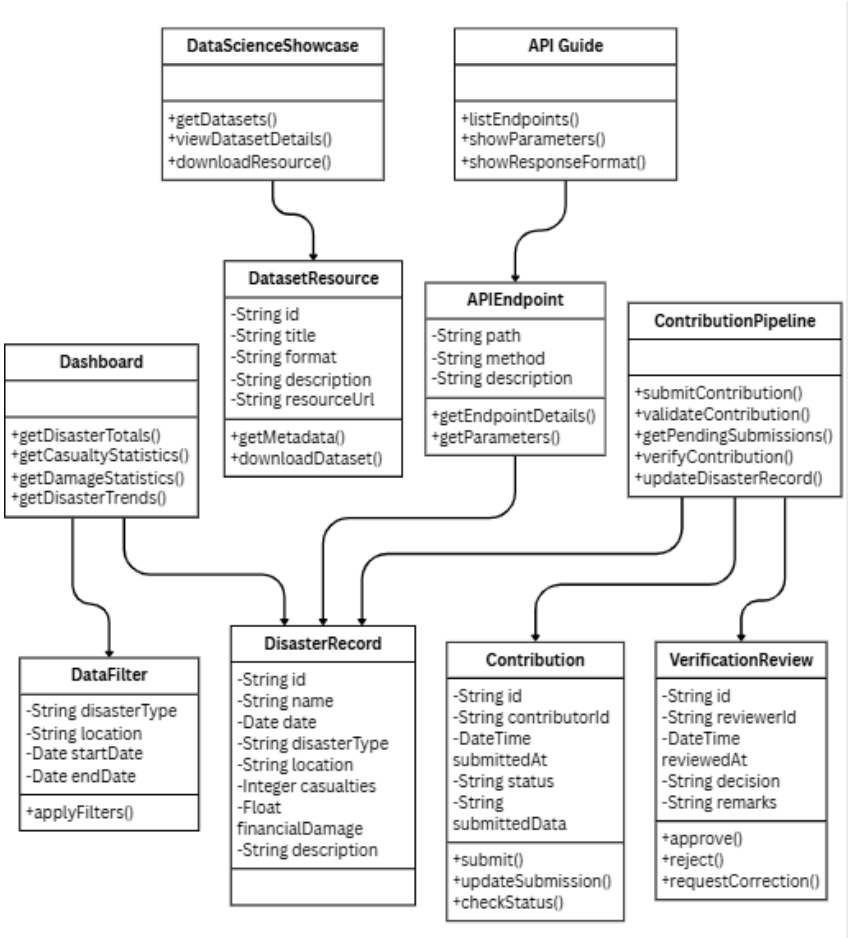


Figure 6: Class Diagram for SAKUNA.PH Enhancements

ii. Entity-Relationship Diagrams

The system processes data across three primary data storage models, conceptualized into distinct entities with defined attributes, primary keys, and cross-domain relationship constraints:

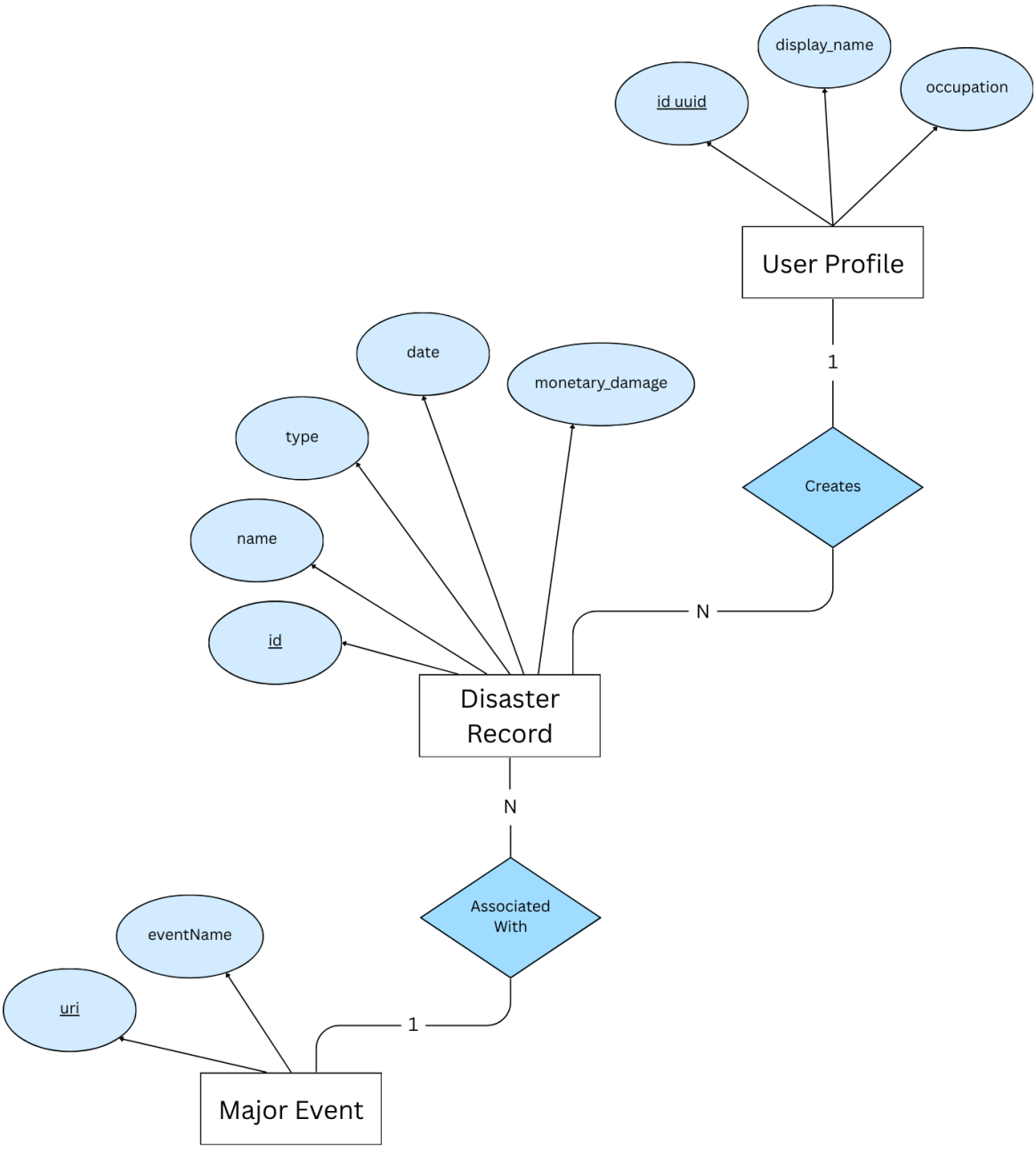


Figure 7: Conceptual Entity-Relationship Diagram

1. Disaster Record Entity (disaster_data.csv)

- **Entity Type & Format:** Flat-file tabular data object (60 KB comma-separated values file containing 525 records spanning 2011–2026, imported at build time via Rollup and bundled directly into the client application).
- **Relationship Constraints:** Participates in a 1:N relationship with the *Major Event* semantic ontology entity, where multiple structured records map to corresponding RDF event nodes.
- **Attributes & Schema Fields:**
    - id (string, **Primary Key**): Generated unique identifier for each disaster entry.
    - name (string): Official designation or name of the disaster event.
    - type (string): Categorization (e.g., typhoon, earthquake, flood).
    - region, province, city_municipality (string): Hierarchical geographic classification identifiers.
    - date (string/timestamp): Date of occurrence.
    - monetary_damage (float): Quantified economic loss in PHP.
    - affected_families, casualties, injured, missing (integer): Quantitative human impact indicators.

2. User Profile Entity (profiles Table Schema)

- **Entity Type & Format:** Relational database table managed via Supabase BaaS and secured under Row-Level Security (RLS).
- **Relationship Constraints:** Maintains a 1:1 relationship with the external Supabase authentication schema (auth.users), and a 1:N relationship with system interactions, tracking user account ownership over contributions and profile modifications.
- **Attributes & Schema Fields:**
    - id (uuid, **Primary Key / Foreign Key**): Unique user identifier mapping directly to the core Supabase Auth record (auth.users).
    - display_name (string): User's preferred display name.
    - birthdate (date): User's date of birth.
    - city_location (string): Residential city.
    - affiliation (string): Academic or professional organization.
    - occupation (string): User's profession.
    - intended_usage (string): Stated purpose for using the platform.
    - updated_at (timestamp): Tracks the last modification time of the profile row.

3. Major Event Entity (External RDF Graph Ontology)

- **Entity Type & Format:** Semantic triple store hosted externally at sakunagraph.upcsweb.dev.
- **Relationship Constraints:** Linked via ontological URI references to structured data instances, executing dynamic queries over HTTP via SPARQL.
- **Attributes & Schema Fields:**

- uri (string, **Primary Key**): Subject URI using the https://sakuna.ph/ namespace representing a unique semantic node.
- eventName (string): Ontological property mapping to :eventName.
- startDate (date): Ontological temporal property mapping to :startDate.

iii. Data Flow Diagrams

The data processing pipeline is visualized through the following data flow hierarchy:

Level 0 (Context Diagram): Illustrates users interacting with the SvelteKit frontend SPA, authenticating via Supabase Google OAuth, and querying historical disaster information.

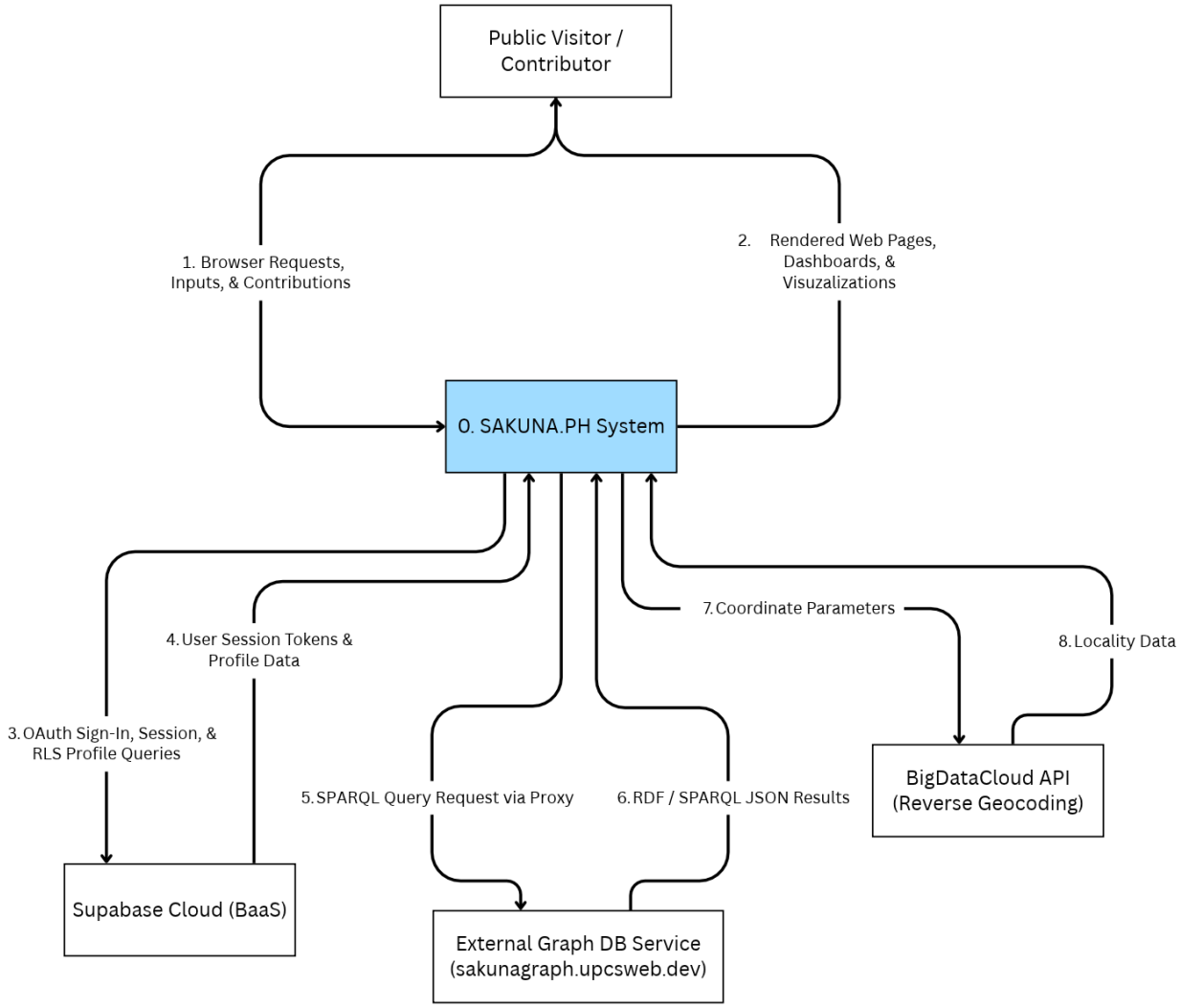


Figure 8: Level 0 Data Flow Diagram (Context Diagram)

Level 1 (Process Breakdown): Details the separation between client-side reactive filtering of bundled CSV/GeoJSON assets, serverless SSR session handling via hooks.server.ts, and server-side proxying of SPARQL queries.

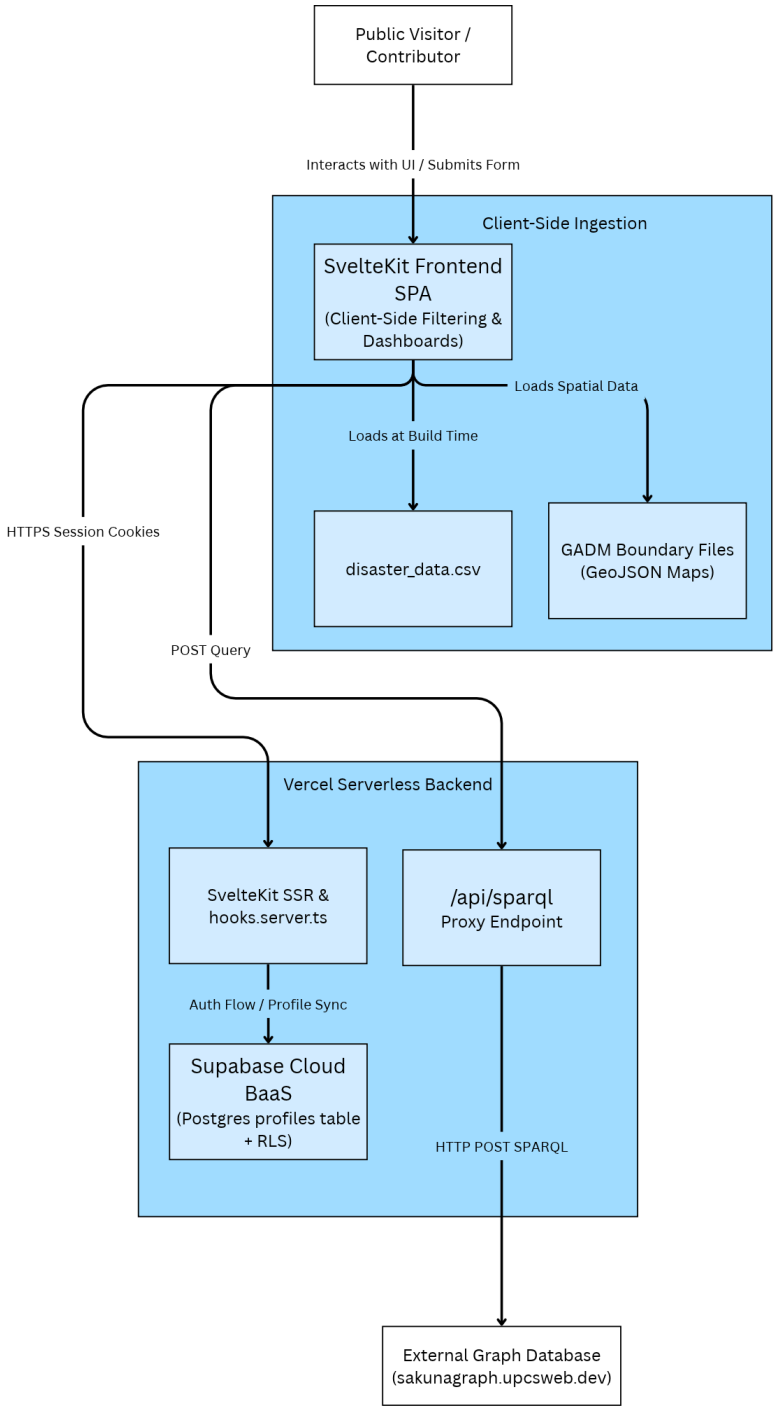


Figure 9: Level 1 Data Flow Diagram (Process Breakdown)

Level 2 (Detailed Data Pipeline): Traces data transformation from build-time Rollup CSV ingestion into Svelte 5 reactive rune states ($derived.by), rendering across D3, LayerCake, and Regl WebGL visualization layers, and asynchronous persistence to Supabase under RLS.

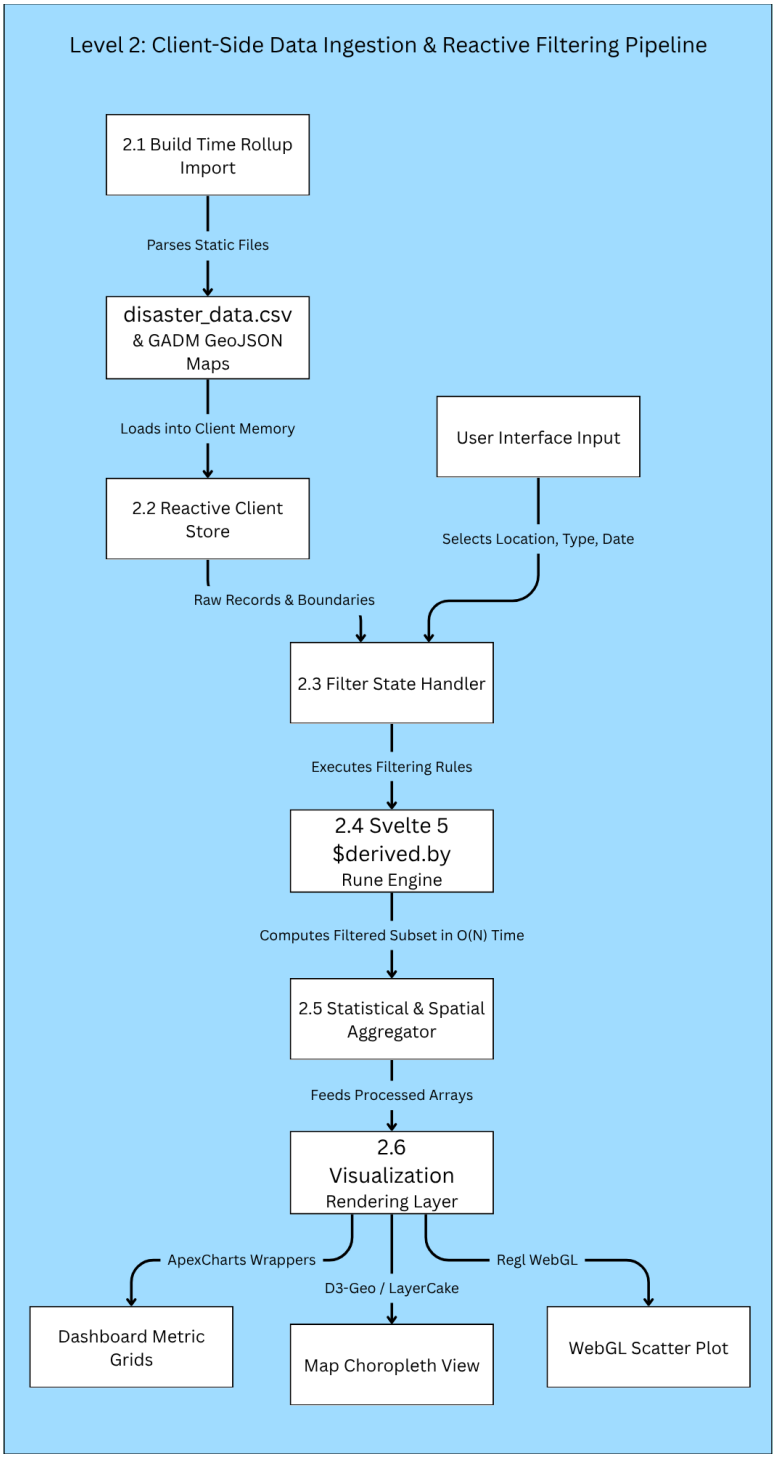


Figure 10: Level 2 Data Flow Diagram (Detailed Client Filtering Pipeline)

c. Component-level design

System components are organized into a modular SvelteKit full-stack architecture. Client-side features (including the modified landing page, contribution pipeline interface, data science showcase, and API guide) are implemented as reactive Svelte components, while backend logic is encapsulated within SvelteKit server routes and hooks. Interfaces are explicitly typed via TypeScript contracts and shared data types.

Commercial-Off-The-Shelf (COTS) and Managed Components:

- **Supabase BaaS:** Utilized for user authentication (Google OAuth with PKCE cookie sessions) and backend profile persistence under Row-Level Security (RLS).
- **Ontotext GraphDB (External Service):** Accessed via a secure server-side proxy route (/api/sparql) to query external disaster ontology triples.
- **BigDataCloud API:** Integrated for client-direct reverse geocoding.
- **Open-Source Visualization & UI Libraries:** D3-geo, LayerCake, regl (WebGL scatter plots), and ApexCharts via Flowbite wrappers.

i. Algorithms

The application relies on efficient client-side data transformation and reactive state management algorithms rather than heavy backend worker queues.

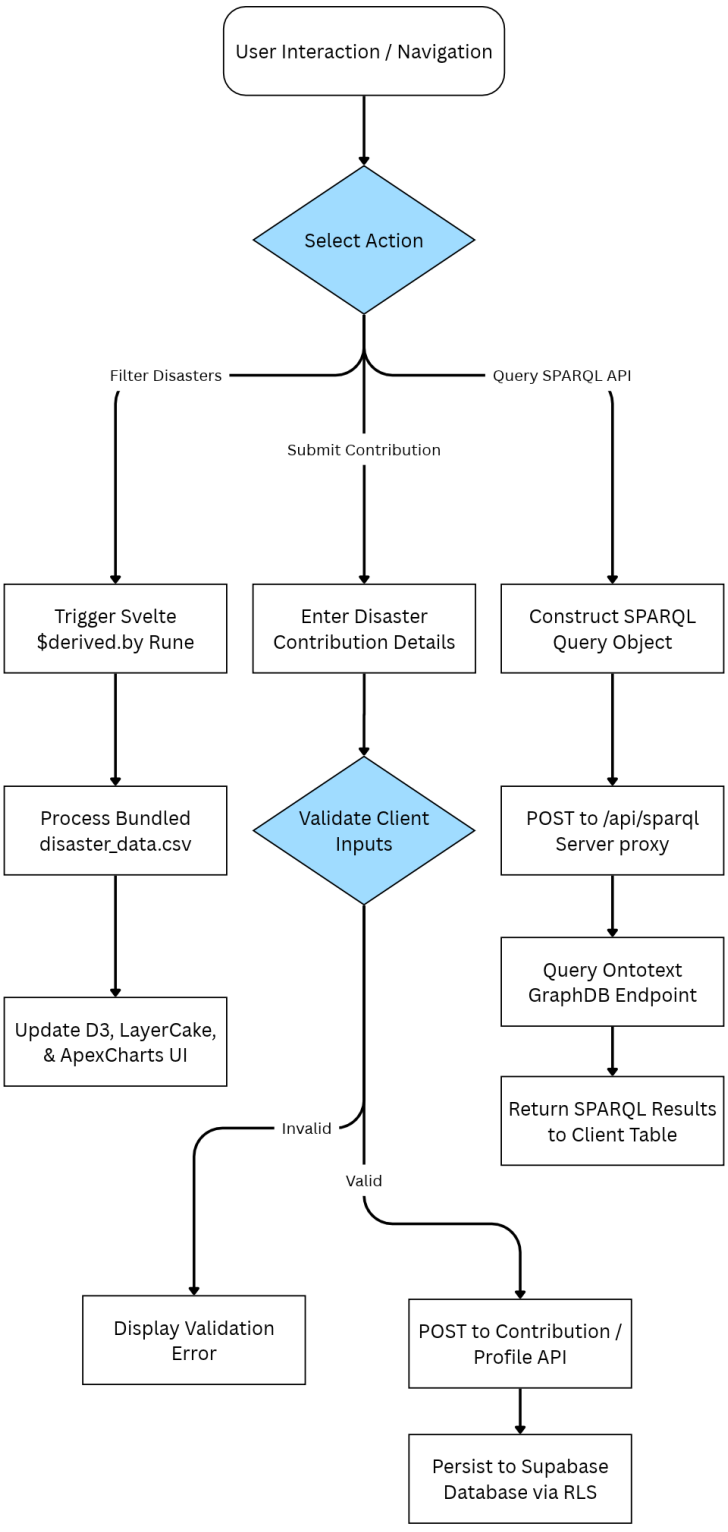


Figure 11: Client Filtering & Contribution Flow

Key Algorithmic Details:

Reactive Filtering Pipeline (filteredData.svelte.ts): Implements an $O(N)$ filtering algorithm using Svelte 5 $derived.by runes that evaluates selected municipal names, disaster categories, and date bounds against the bundled dataset in real time.

Spatial Quadtree Indexing (d3-quadtree): Used in the WebGL scatter plot and map views to index geographical coordinates, providing $O(log\ N)$ nearest-neighbor lookups during user hover interactions.

ii. Data Structures

The system employs several optimized data structures tailored for high-performance client-side rendering and semantic graph querying:

- **Flat Tabular Arrays:** JavaScript object arrays parsed from the build-time bundled CSV (disaster_data.csv, 525 rows). Chosen for maximum iteration and filtering performance across dashboard metric calculations without incurring network fetch overhead.
- **GeoJSON Feature Collections:** Hierarchical tree-based spatial structures (gadm41_PHL_1.json, gadm41_PHL_2.json) loaded into client memory to render precise administrative boundary polygons for choropleth maps.
- **RDF Triples (Subject-Predicate-Object):** Graph-based semantic triples managed by the external SPARQL endpoint (sakunagraph.upcsweb.dev) and queried through the /api/sparql proxy to represent complex ontological disaster relationships.
- **Relational Profile Records:** Structured JSON objects managed through Supabase and mapped to user session tokens for account customization and contribution tracking.

d. Interface elements / Usability design

SAKUNA.PH uses a web-based interface that allows users to explore disaster records, view geographical and statistical information, and access data contribution and documentation features. The interface design emphasizes consistent navigation, readable information presentation, and straightforward interactions. The following subsections describe the navigation mechanisms, visual design, and expected user interaction flows.

i. Interface Design - Menus and links

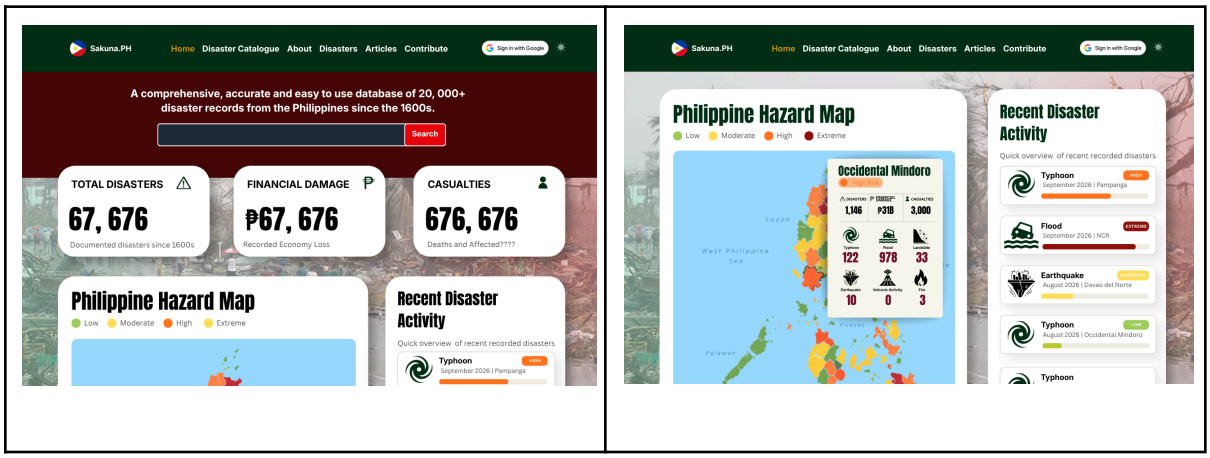


Figure 12. Landing Page UI Mockup

SAKUNA.PH uses a horizontal navigation menu at the top of the landing page to provide access to its main sections. The menu includes Home, Disaster Catalogue, About, Disasters, Articles, and Contribute, along with Google sign-in and a theme control. Each menu item is clearly labeled to help users identify its purpose and navigate to the corresponding section.

The landing page also provides a search bar that allows users to search for disaster records. Below it, summary cards display key information, including total disasters, financial damage, and casualties. The Philippine Hazard Map presents the geographical distribution of hazards, while the Recent Disaster Activity section provides a quick overview of recently recorded disasters. These elements allow users to access important disaster information directly from the landing page.

The navigation will also provide access to the newly added Contribution Pipeline, API Guide, and Data Science Projects Showcase. These features will be incorporated into the existing navigation structure so users can discover and access them without navigating through unrelated sections.

The interface design promotes usability through descriptive menu labels, consistent navigation placement, and a clear visual hierarchy. Frequently accessed information is presented directly on the landing page, reducing the steps required to explore disaster records. The search function provides an additional way to locate information, while the hazard map and summary cards support users who prefer visual representations of disaster data.

ii. Aesthetic Design- Layout and graphics

The interface uses a consistent visual style to distinguish navigation, content, and interactive elements. The header uses a dark green background, while maroon and dark red sections emphasize headings and important content areas. Light-colored backgrounds and white content cards separate information into readable sections.

The landing page follows a dashboard-style layout. Summary cards present key disaster statistics, while the Philippine Hazard Map provides a geographical representation of hazards. The Recent Disaster Activity panel displays individual disaster entries in a compact list. This arrangement allows users to scan important information without having to open separate pages for every detail.

The multi-step contribution form follows the same general visual style as the landing page. It groups related fields into steps and uses a progress indicator to show the user's position in the submission process. Disaster-type images help users identify the appropriate contribution category, while clearly labeled fields guide data entry. Figure 2 presents the proposed contribution interface.

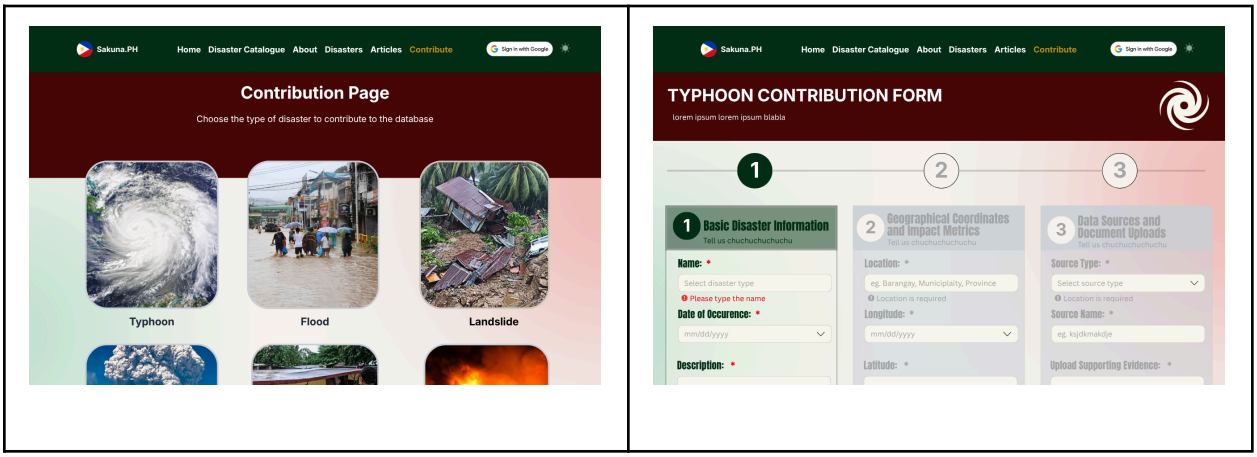


Figure 13: Multi-Step Data Contribution Form

The interface also uses graphical elements to communicate information. The hazard map uses different colors to distinguish hazard levels, and icons and summary cards help users recognize information categories. Text labels accompany important visual indicators so that meaning does not depend on color alone.

The design supports usability through consistent page structure, visual grouping, recognizable controls, and readable content. Form labels, field requirements, and validation messages should be presented clearly to reduce input errors. The layout should also remain usable at different screen sizes, and text and interactive elements should maintain sufficient contrast and clear visual focus.

iii. Navigation Design

The navigation design is organized into semantic units, which group related interactions into meaningful tasks. These units allow users to complete common activities without navigating through unnecessary pages.

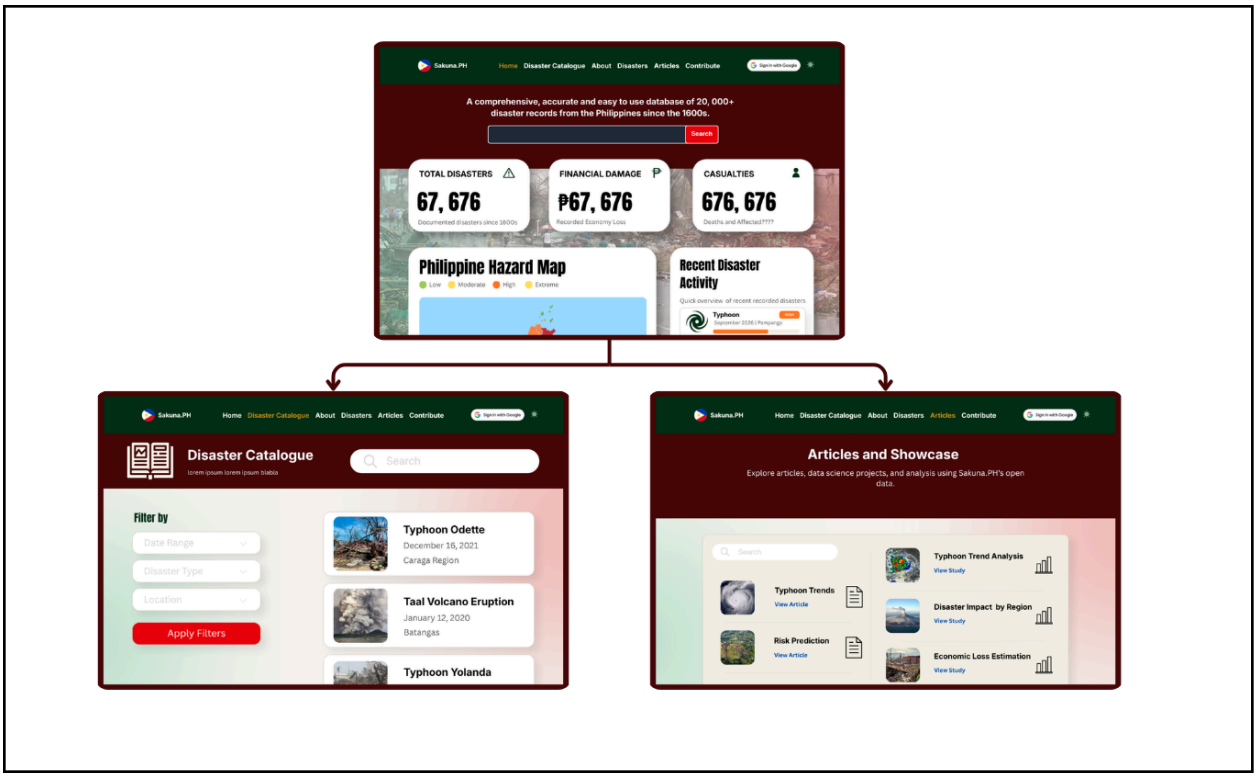


Figure 14: Site Navigation and Site Map Diagram

Three typical usage scenarios are considered to evaluate the navigation design.

Scenario 1: Finding a Disaster Record

The user starts at the landing page, opens the Disaster Catalogue, enters a search query, and selects a matching record. Assuming the user begins on the landing page, this task requires approximately three mouse clicks and 15 keystrokes. Of the 18 total interactions, 3 (16.7%) are navigation actions, while 15 (83.3%) are data-entry actions.

Scenario 2: Accessing the Articles and Showcase Section

The user starts at the landing page and selects the Articles and Showcase section using the main navigation menu. The user then selects an article or project to view its details. This task requires approximately two mouse clicks, both of which are navigation actions. Therefore, 100% of the counted interactions are navigation actions.

Scenario 3: Submitting a Disaster Contribution

The user opens the contribution page, selects a disaster type, and proceeds through the multi-step contribution form. The user enters the required disaster details, reviews the information, and submits the contribution. Assuming three navigation clicks, five field-selection clicks, 60 keystrokes, and one submission click, the task requires approximately 69 interactions. Of these, 3 (4.3%) are navigation actions, while the remaining 66 (95.7%) involve form interaction, data entry, and submission.

The interaction counts are estimates based on the proposed interface and may vary depending on the number of required fields, the user's input, and the final implementation. Scrolling, validation errors, and additional confirmation steps are excluded from the estimates.

Semantic Unit	**User Goal**	**Main Interaction Sequence**
Disaster search	Find a particular disaster.	Landing Page -> Disaster Catalogue -> Search -> Select Record
Content exploration	Read an article or view a project.	Landing Page -> Articles and Showcase -> Select Item
Disaster contribution	Submit a disaster record.	Landing Page -> Contribute -> Select Type -> Complete Form -> Submit